

CMSC 471 Programming Assignment 1: A* Search and Heuristics with DOGCAT

Name: Toni Graham

Section: 02

Notation used throughout

For a state n and the goal word g of the same length:

- $M(n)$ is the set of positions where n and g hold different letters.
- $d(n) = |M(n)|$ is the number of mismatched positions, so d is also the baseline heuristic h_0 .
- $S(c)$ is the Scrabble value of letter c from the table in the assignment.
- $r(w)$ is the rarity value stored for word w in `words34.txt.gz`.

One fact does the heavy lifting in every proof below, so I will state it once and refer back to it as the **one-position-per-action fact**. Every action replaces the letter at exactly one position, and the last word on a solution path is g , which matches the goal in every position. So for each position i in $M(n)$, some action on the path has to write $g[i]$ into position i . Two different positions need two different actions, because a single action only touches one position. That gives two consequences: any path from n to the goal contains at least $d(n)$ actions, and it contains at least $d(n)$ distinct actions each of which writes a specific goal letter into a specific position.

I should also note what the numbers in the dictionary file actually are. The comment in the starter `dc.py` calls them frequencies, but the values run the other way: `the` is 0.016 and `oozy` is 16.35. Higher means rarer, so I treated them as rarity values, matching the $1 + \text{rarity}(w)$ definition in the assignment. This also reproduces the sample output in the handout exactly.

Question 1: Heuristic design and admissibility

1.1 Steps

Formula.

$$h_{\text{steps}}(n) = d(n)$$

What it lower-bounds. Under the steps objective every action costs exactly 1, so the total cost of a path is just the number of actions on it. By the one-position-per-action fact, any path from n to the goal needs at least $d(n)$ actions, so it costs at least $d(n)$.

Why it cannot overestimate. There is no way to fix two mismatched positions with one action, so the count of mismatches is a hard floor on the number of actions, and each action contributes exactly 1. The estimate equals the floor, so it can never exceed the true remaining cost. At the goal $d(n) = 0$, so the heuristic returns 0 as required. It is not the trivial heuristic, since it is strictly positive at every non-goal state.

1.2 Scrabble

Formula.

$$h_{\text{scrabble}}(n) = \text{sum of } S(g[i]) \text{ for every } i \text{ in } M(n)$$

In words, look only at the positions that are currently wrong, and add up the Scrabble values of the letters the goal wants in those positions.

What it lower-bounds. Under this objective an action's price is the value of the letter it writes, and it does not depend on which letter is being replaced or on the word being left behind. By the one-position-per-action fact, for each wrong position i there is an action on the path that writes $g[i]$ into position i , and that action costs exactly $S(g[i])$. Those actions are distinct across positions. So the sum is a lower bound on the combined price of a subset of the actions on the path.

Why it cannot overestimate. The sum only counts a subset of the actions the path must contain, and every action not counted still has a positive cost, since the smallest Scrabble value in the table is 1. Dropping those extra actions can only make the estimate smaller than the truth, never larger. A path is also free to overshoot, for example by writing the right letter into a position, changing it away, and writing it back later. Any such detour adds actions and therefore adds cost, so it only widens the gap in the safe direction. Each position is charged at most once, for the one letter the goal actually needs there. At the goal, $M(n)$ is empty and the sum is 0.

This heuristic turns out to be very sharp. For `bear` to `duck` it returns 11, which is the exact optimal cost, so A^* expanded only 6 nodes.

1.3 Frequency

Formula.

$h_freq(n) = 0$ $= (d(n) - 1) + (1 + r(g))$	$\text{if } n == g$ otherwise
--	--

which simplifies to $d(n) + r(g)$ for non-goal states.

What it lower-bounds. Under this objective, entering word w costs $1 + r(w)$, and rarity values are nonnegative, so every single action costs at least 1. Two separate facts combine here. First, the path needs at least $d(n)$ actions, by the one-position-per-action fact. Second, the very last action on any path enters the goal word itself, and that action costs exactly $1 + r(g)$, a quantity I know without knowing anything about the rest of the path. So I charge the known exact price for the final action and the cheapest conceivable price, namely 1, for each of the at least $d(n) - 1$ actions that come before it.

Why it cannot overestimate. Splitting the true cost of any path into the final action plus everything before it, the final action contributes exactly $1 + r(g)$ and the earlier actions contribute at least 1 each, with at least $d(n) - 1$ of them. Summing the two pieces gives exactly the formula, so the formula cannot exceed the true cost. The only assumption is $r(w) \geq 0$ for every word, which holds for this dictionary, where the minimum value is 0.0162 for the . The goal case is handled separately, because $d(n) + r(g)$ would return $r(g)$ rather than 0 at the goal.

Note that this heuristic is only admissible because the rarity term is applied once. Applying it once per mismatch breaks admissibility, which is exactly the mistake I audit in Question 4.

1.4 Verification of all three

The `deltas` that `dcsolve.py` prints were non-positive on every problem I ran, but as the assignment points out, that only samples the states on one returned path. To get real evidence I wrote a checker (`verify.py`) that runs Dijkstra backwards from the goal across the whole word graph. That produces the true optimal remaining cost $h^*(x)$ for every state x that can reach the goal, so I can compare $h(x)$ against $h^*(x)$ directly instead of inferring anything from a single path. Across 54 goal words and all three cost measures, that is 455,136 (state, goal) pairs, and it found zero overestimates.

The backward search has to be set up carefully, because the graph is undirected in structure but its edge weights are not symmetric (see Question 3). The cost of an action depends on the word being entered, so when relaxing an edge outward from the goal I charge the cost of the forward action, not the backward one.

Question 2: Does the heuristic actually help?

Node counts are the values reported by the supplied search functions with `display=True`. Every search ran on a freshly constructed `DC` instance.

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5.00	773	5.00	16	97.9%
WHY → ASK	Scrabble	9.00	765	9.00	13	98.3%
WHY → ASK	Frequency	38.855	808	38.855	634	21.5%
BEAR → DUCK	Steps	4.00	691	4.00	6	99.1%
BEAR → DUCK	Scrabble	11.00	2981	11.00	6	99.8%
BEAR → DUCK	Frequency	38.240	1084	38.240	302	72.1%

1. Did UCS and A* always return the same optimal cost? Yes, in all six runs the two costs agreed exactly, so there is no discrepancy to investigate. This is what I expected, since UCS is optimal for nonnegative costs and A *with an admissible heuristic is optimal too, so the two have to agree on the optimal cost even when they return different paths. It is a useful check, because an inadmissible heuristic usually shows up here as A reporting a cost above the UCS cost.*

2. Largest percentage reduction. For both word pairs it was the Scrabble measure: 98.3% for WHY → ASK and 99.8% for BEAR → DUCK.

3. Least informative heuristic. For both word pairs it was the frequency measure, at 21.5% and 72.1% respectively. It still helps, but by a much smaller margin than the other two.

4. Explaining the differences. The useful quantity is how much of the true optimal cost each heuristic actually claims at the start state, since that is what lets A* rule out bad branches early:

Word pair	Cost measure	h(start)	Optimal cost	Fraction claimed
WHY → ASK	Steps	3.000	5.000	0.60
WHY → ASK	Scrabble	7.000	9.000	0.78
WHY → ASK	Frequency	7.233	38.855	0.19
BEAR → DUCK	Steps	4.000	4.000	1.00
BEAR → DUCK	Scrabble	11.000	11.000	1.00
BEAR → DUCK	Frequency	11.230	38.240	0.29

For BEAR → DUCK the steps and Scrabble heuristics are exactly right at the start, claiming the full optimal cost. That is because the optimal path fixes each of the four wrong positions exactly once and never revisits one, which is precisely the scenario both heuristics assume. When a heuristic is exact, A* walks more or less straight to the answer, and indeed it expanded only 6 nodes in both cases.

The frequency heuristic is structurally weaker, and the reason is visible in its definition. It charges the bare minimum of 1 for every action except the last, because all it can safely assume about an intermediate word is that its rarity is nonnegative. But the intermediate words on these actual paths are obscure, with rarities in the 7 to 16 range, so each real action costs somewhere around 5 to 17 rather than the 1 the heuristic assumes. For WHY → ASK the seven actions cost 38.855 in total, an average of about 5.55 each, while the heuristic only commits to 1 each plus a single 4.23 for ask . It ends up claiming under a fifth of the true cost, which leaves A* expanding almost as many nodes as UCS.

So the gap is not about runtime or implementation detail. It comes from how much the cost structure lets you prove in advance. Under steps and Scrabble, the price of fixing a given position is fixed and knowable from the goal word alone, so the heuristic can charge the full amount. Under frequency, the price of an action depends on which word you land on, and that word is not determined until you choose a path, so the only safe floor is the trivial one. That is also why the frequency heuristic does relatively better on BEAR → DUCK (0.29) than on WHY → ASK (0.19): the BEAR → DUCK optimal path is shorter, so there are fewer unknown intermediate words being undercharged.

Question 3: Reversibility and direction-dependent costs

The distinction that matters here is between the structure of the graph and the weights on its edges. Reversibility is a statement about structure: if w_1 can legally become w_2 , then w_2 can legally become w_1 , so the same set of paths exists in both directions and a problem is solvable one way

exactly when it is solvable the other way. That says nothing about price. The cost of an action in this problem is attached to what you are moving *into*, either the letter you write or the word you enter, and that changes when you traverse the same path backwards. So the graph is undirected in structure while carrying direction-dependent weights.

Steps: yes, always symmetric

Take an optimal path from w_1 to w_2 with k actions. Reversing it gives a legal path from w_2 to w_1 , also with k actions, because each individual action is reversible. Under steps, cost equals the number of actions, so that reversed path costs k as well. This means the optimal cost from w_2 to w_1 is at most the optimal cost from w_1 to w_2 , and the identical argument in the other direction gives the reverse inequality. The two are therefore equal. My runs confirm this: 5 and 5 for WHY and ASK, 3 and 3 for DOG and CAT, 4 and 4 for BEAR and DUCK, 3 and 3 for KIT and ZAP.

Scrabble: no

Counterexample. bear to duck costs 11, while duck to bear costs 6.

Both directions use the same four-action path, so the step count is identical. The costs differ because each direction pays for a different set of letters. Going forward, the letters written are k, c, u, d :

```
bear -> beak   write k    5
beak -> beck   write c    3
beck -> buck   write u    1
buck -> duck   write d    2   total 11
```

Going backward, the letters written are b, e, a, r :

```
duck -> buck   write b    3
buck -> beck   write e    1
beck -> beak   write a    1
beak -> bear   write r    1   total 6
```

The general point is that traversing a path forward writes the letters of the destination word into the positions where the two words differ, while traversing it backward writes the letters of the origin word into those same positions. So a fixed path costs the total Scrabble value of the destination's distinguishing letters, and reversing it costs the total value of the origin's. Those agree only by coincidence. *duck* is built from expensive letters (d is 2, c is 3, k is 5) while *bear* is built from cheap ones (b is 3, and e, a, r are 1 each), so the asymmetry is large. *kit* and *zap* show the

same thing, 14 forward against 7 backward, and so do why and ask at 9 against 14. Symmetry can happen by accident: dog to cat is 5 in both directions, because c, a, t and d, o, g both happen to total 5.

Frequency: no

Counterexample. dog to cat costs 23.022, while cat to dog costs 21.955. Both use the path through don, dan, can.

The reason is cleaner than in the Scrabble case. On any fixed path you pay $1 + r(w)$ for every word you enter, which is every word on the path except the one you start from. Walking forward you pay for $r(\text{cat})$ and never pay for $r(\text{dog})$; walking backward you pay for $r(\text{dog})$ and never pay for $r(\text{cat})$. Every other word on the path is charged in both directions. So the two costs differ by exactly $r(\text{cat}) - r(\text{dog})$, which is $6.886906 - 5.819885 = 1.067021$, and that matches the observed difference of $23.022 - 21.955$ exactly. The costs can only agree when the two endpoints happen to have identical rarity, which essentially never happens here. All four word pairs I tested came out asymmetric under frequency.

Question 4: LLM heuristic audit

4.1 Model, prompt, and proposal

Model used: Claude Opus 5, accessed through Cursor.

Exact prompt given:

In DOGCAT, the initial and goal states are lowercase words of the same length, three or four letters, and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By "more informative," we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Proposed heuristic. The model proposed charging the goal word's rarity once for every mismatched position:

$$h_{llm}(n) = d(n) * (1 + r(g))$$

Justification the model gave. Each of the $d(n)$ mismatched positions needs its own action, since one action only changes one letter. Every action costs $1 + \text{rarity}$ of the word it enters. So instead of charging just 1 per action as h_0 effectively does, charge $1 + r(g)$ per action, using the goal's rarity as a stand-in for the rarity of the words along the way.

It is worth saying that the proposal does satisfy the assignment's "more informative" condition. Since rarity is nonnegative, $1 + r(g) \geq 1$, so $h_{llm}(n) \geq d(n) = h_0(n)$ at every state, with strict inequality at every non-goal state whenever $r(g) > 0$. The problem is the other half of the requirement.

4.2 Is it admissible? No

Counterexample. Goal `maks`, with $r(\text{maks}) = 15.762400$. State `jake`, which differs from `maks` in two positions (`j` against `m`, and `e` against `s`), so $d = 2$.

$$h_{llm}(\text{jake}) = 2 * (1 + 15.762400) = 33.524800$$

The true optimal remaining cost is much lower. The path `jake` $\rightarrow$ `make` $\rightarrow$ `maks` costs:

```
enter make:  1 + 1.903091  =  2.903091
enter maks:  1 + 15.762400 = 16.762400
-----
                        19.665491
```

I confirmed by backward Dijkstra over the whole graph that 19.665491 really is the optimal cost from `jake` to `maks`, so h_{llm} overestimates by 13.859309. The heuristic is inadmissible, which means A^* using it is not guaranteed to return an optimal solution.

Why it fails in general, not just here. The flaw is treating $r(g)$ as representative of every word on the path when it is only guaranteed for one of them. The final action is the only action guaranteed to enter the goal word. Every earlier action enters some other dictionary word, and that word can be far more common than the goal. In the counterexample, `make` has rarity 1.90 against the goal's 15.76, and the dictionary contains words as common as `the` at 0.016. So whenever the intermediate words on the optimal path are collectively more common than the goal, specifically

when their rarities sum to less than $(d - 1) * r(g)$, the proposal overcharges. Since rare goal words are exactly the interesting case, this is a common situation rather than a freak one.

An important observation about testing. When I ran A* with h_{llm} on the two frequency problems from Question 2, it returned the correct optimal cost both times, 38.855 and 38.240, and it expanded dramatically fewer nodes than any admissible alternative (306 and 12). If I had only looked at those two problems, I would have wrongly concluded the heuristic was both safe and excellent. It is neither. It just happened not to overestimate anywhere that mattered on those particular instances. This is precisely the point the assignment makes: tests can expose counterexamples but cannot establish admissibility.

4.3 The repair

Minimal repair. Charge the goal's rarity once instead of d times:

```
h_repaired(n) = 0                                if n == g
               = (d(n) - 1) + (1 + r(g)) otherwise
```

This is the smallest change that fixes the problem, and it is the heuristic I implemented in `dc.py`. The proof is in Question 1.3 above: the last action is the one that enters the goal, and it is the only one whose cost I can pin down, so it gets the $1 + r(g)$ charge while every other action gets the safe floor of 1. It still satisfies the "more informative" condition relative to h_0 , since $h_{\text{repaired}}(n) = d(n) + r(g) \geq h_0(n)$, strictly greater at every non-goal state when $r(g) > 0$.

An additional tightening. Since the repaired version gives up a lot by charging only 1 per intermediate action, I also built a tighter variant that recovers some of it. When two or more positions are wrong, the next-to-last word on any path has to be a legal neighbour of the goal, one action away from it. So I can charge the rarity of the cheapest such neighbour in addition to the goal's own:

```
floor(g) = min over all dictionary neighbours v of g of r(v)

h_tighter(n) = 0                                if d(n) == 0
              = 1 + r(g)                        if d(n) == 1
              = (d(n) - 2) + (1 + floor(g)) + (1 + r(g)) otherwise
```

The argument is the same shape as before, just applied to the last two actions instead of only the last one. The path needs at least $d(n)$ actions; the final one enters g at cost exactly $1 + r(g)$; the one before it enters a neighbour of g , so it costs at least $1 + \text{floor}(g)$; and the remaining $d(n) - 2$ or more actions cost at least 1 each.

4.4 Evidence on the two frequency problems

All searches ran on fresh problem instances, with node counts from `display=True`.

Word pair	Heuristic	Cost returned	Nodes expanded	Admissible?
WHY → ASK	UCS, no heuristic	38.855	808	n/a
WHY → ASK	h_0 (mismatch count)	38.855	755	yes
WHY → ASK	h_{llm} (proposed)	38.855	306	no
WHY → ASK	$h_{repaired}$	38.855	634	yes
WHY → ASK	$h_{tighter}$	38.855	295	yes
BEAR → DUCK	UCS, no heuristic	38.240	1084	n/a
BEAR → DUCK	h_0 (mismatch count)	38.240	720	yes
BEAR → DUCK	h_{llm} (proposed)	38.240	12	no
BEAR → DUCK	$h_{repaired}$	38.240	302	yes
BEAR → DUCK	$h_{tighter}$	38.240	127	yes

Every row returned the same optimal cost, including the inadmissible one.

I also stress-tested the two safe heuristics with the backward Dijkstra checker across 34 goal words, covering 96,943 states under the frequency measure, and found no overestimates for either.

4.5 Conclusion

The evidence does not support using the model's proposal as written. It is inadmissible, and I have both a concrete counterexample and a general explanation of why the error occurs, so this is not a borderline case. The fact that it produced correct answers on both test problems while expanding the fewest nodes of anything I tried is a warning about the method, not a defence of the heuristic. Its speed comes directly from the overcharging that makes it unsafe, and on a different instance that same overcharging would prune the optimal path.

The evidence does support the repaired versions. Against the h_0 baseline, $h_{repaired}$ cuts expansions from 755 to 634 on WHY → ASK and from 720 to 302 on BEAR → DUCK, and $h_{tighter}$ cuts them further to 295 and 127. Both keep the optimal cost and both survived the exhaustive check. So the model's underlying instinct, that the frequency objective lets you say more than a plain mismatch count, was right. What it got wrong was the scope of the claim, applying a guarantee that holds for one action to all of them.

I deliberately shipped $h_{repaired}$ rather than $h_{tighter}$ in `dc.py`. The question asked for the smallest repair that restores admissibility, and $h_{repaired}$ is exactly that: it changes one thing, the

number of times the goal's rarity is charged. `h_tighter` is a different heuristic rather than a repair of the proposal, since it brings in information the proposal never used, namely the rarity of the goal's neighbours. Keeping the submitted version minimal also keeps it simple enough to reason about in one sitting, which matters more to me than the node counts, given that both versions return the same optimal costs. `h_tighter` is included here because it shows how much headroom the minimal repair leaves on the table, and it would be a drop-in replacement for the `h` method if faster search were the goal.

Generative AI disclosure

- **System used:** Claude Opus 5.
- **What I used it for:** Question 4 requires it. I also used it while working on the implementation and this write-up, for discussing heuristic designs and for drafting the verification scripts described below.
- **How I verified the results:** I did not take the heuristic claims on faith, which is how the Question 4 error was caught. I wrote a checker that computes the true optimal remaining cost for every reachable state by running Dijkstra backwards from the goal, then compares the heuristic against it. That confirmed all three heuristics in `dc.py` on 455,136 (state, goal) pairs with zero overestimates, and it is the same tool that found the `jake` to `maks` counterexample. I separately confirmed that A* agrees with uniform-cost search on the optimal cost in all six Question 2 runs, and that my implementation reproduces every sample output in the assignment handout exactly, including the `deltas` values. I also wrote out proofs by hand for each heuristic rather than relying only on the experiments, since passing tests does not establish admissibility.

Supporting files

These are development tools and are not part of the submission. `dc.py` does not depend on any of them.

- `verify.py` checks admissibility of all three heuristics against true optimal costs via backward Dijkstra.
- `experiment.py` generates the Question 2 table.
- `q34.py` generates the Question 3 direction comparisons and the Question 4 audit table.
- `q4detail.py` searches for the most hand-checkable counterexample to the proposed heuristic and stress-tests the repairs.
- `hstart.py` prints the `h(start)` against optimal cost figures used in Question 2.